

通往帝国之路👑

AI Native 时代的战略觉醒与能力跃迁

Palantir · 看清终局

Skills/Ontology · 掌握机制

OpenClaw · 拿到火种

从 Ontology 思维到 Agent 架构 — 构建我们自己的 AI 战斗操作系统

探地图 · 升时代 · 造兵器

2026.03

想象你正在打一局AI地图的「帝国时代」

AI 竞赛和即时战略游戏的底层逻辑完全一致



探地图

Scout the Map

看清 AI 世界正在发生什么

黑雾下的地图 = 你不了解的 AI 前沿。不探索，你以为世界还在「Chat 时代」；探完地图才发现——Palantir 已经在用 Ontology 重构企业操作系统，Agent 已经在替人 7×24 工作。

→ Palantir：看清 AI Native 的终局形态



升时代

Advance to Next Age

从石器时代进化到帝国时代

石器时代 = 手动写 Prompt

封建时代 = 用 Copilot 辅助

帝国时代 = 构建自己的 Agent 体系

升时代需要理解底层机制——Skills 的三层渐进加载 + MCP 工具协议，就是你的「升级科技树」。

→ Skills：掌握 AI 的底层运行机制



造兵器

Build Your Arsenal

打造真正的 AI 战斗力

光看不练是假把式。OpenClaw 是一套现成的 AI Native 架构，开源、可跑、可复制——Channel + Agent + Skills + Memory 的四层设计是我们快速借鉴、构建自己 AI 能力的最佳蓝本。

→ OpenClaw：可直接借鉴的 Agent 架构

→ Claude&Opus4.6：最强的模型和编程工具

你现在处于哪个时代？



简单套壳提示词
用落后/免费模型
没有工作流、没有记忆

把 AI 当搜索引擎用。每次对话从零开始，
Prompt 写了又丢。用最便宜的模型，得到最廉价的输出。

你以为在用 AI，其实 AI 在用你的时间。

⚠ 大多数人还在这里



掌握 Skills 体系
使用较好模型
有复用、有工作流

知道用 Skills 把经验固化为可复用模块，选择
Claude/GPT 等前沿模型。AI 开始成为真正的助手，不再是一次性工具。

效率提升显著，但还是「人驱动 AI」。

↑ 少数先行者已经到达



本体语义 + Skills + 最强模型
构建 AI 战斗操作系统
AI 驱动决策、自主行动

用 Ontology 思维建立业务数字孪生，
Skills+MCP 编排完整工作流，配合最强模型。AI
不再是助手——而是你的决策操作系统。

「AI 驱动人」的降维打击。

★ 今天分享的目标：带大家到这里
Anthropic&Palantir

这个时代最强的武器 已经握在我们每个人手中

⚡ Claude Code + Opus 4.6



内置工具体系

代码执行 · 文件操作 · Web 搜索 · 终端控制 · 多模态理解——武器库已满配



Skills 语义编程

可复用能力模块 · 渐进加载 · 跨平台——把每个人的经验固化为 AI 超能力



Opus 4.6 最强大脑

当前地球最强推理模型——深度思考 · 复杂规划 · 精准执行的终极引擎

This is how I feel when Claude is down:

#Claude



当全员装备 Claude Code + Opus 4.6 + Skills

每一个人，都是一支军队。



帝国时代

IMPERIAL AGE

AI 正在从「对话工具」 变成「操作系统」

三个视角，看懂 AI 落地的底层逻辑

01



Palantir — 真正的 AI Native

Ontology 是灵魂：数据→语义→决策→行动的完整闭环

不是给业务加 AI，是业务本身运行在 AI 之上。理解 Ontology，就理解了下一代企业操作系统。

02



Skills — AI 的底层运行机制

从 Prompt Engineering 到 Context Engineering 的范式跃迁

Skills 教 AI 「怎么做」，MCP 让 AI 「能连接」。两者结合，AI 从「会聊」变成「会干」。

03



OpenClaw — 可快速借鉴的 AI Native 架构

Channel · Agent · Skills · Memory 四层架构，即学即用

最佳的 AI Native 开源蓝本。不用从零造轮子——直接借鉴它的架构设计，构建我们自己的 AI 能力。

它们之间的深层联系

Palantir 定义了终局、Skills 是必须拥抱的基建、结合本体才能构建操作系统



终局：AI Native 企业 = Ontology 型操作系统

Palantir 证明：AI 必须嵌入业务语义层才能产生价值。Objects + Links + Actions 构成数字孪生，AI 在其上推理并直接触发业务操作——而非停留在聊天。



基建：Skills & Ontology = 语义编程，AI 最灵活的基础设施

Skills 本质上是语义编程——用自然语言定义程序性知识，让 AI 像老员工一样执行专业流程。这是 AI 时代最灵活的基建之一，必须充分拥抱。但 Skills 单独只是「能力」，结合 Ontology 本体语义才能升级为「决策系统」。



路径：OpenClaw = 我们可以快速借鉴的 AI Native 蓝本

四层架构（Channel→Agent→Skills→Memory）是最优雅的 AI Native 设计。开源可审计、架构清晰、264K Stars 验证——不需要从零发明，直接站在巨人肩膀上构建。



一个统一公式

感知 → 理解 → 推理 → 行动 → 学习

Palantir 的实现

数据管道 → Ontology 语义 → AIP 推理 → Actions 执行 → 飞轮迭代

Skills（语义编程）——充分拥抱的基建

渐进加载 → 语义编程定义流程 → LLM 推理 → MCP 工具调用
→ 结合本体语义，从「能力」升级为「决策系统」

OpenClaw 的实现（可直接借鉴）

Channel 适配 → Agent 大脑 → Skill 双手 → Memory 档案柜

Skills 是 AI 时代的语义编程基建——充分拥抱

结合本体语义，能力才能升级为操作系统

OpenClaw 的架构是我们最快的落地起点。

未来属于那些「看得见」的人

01

Ontology 是关键——通往帝国之路

Skills 是基建，模型是引擎，但 Ontology 才是把一切串联成操作系统的灵魂。用本体语义定义业务的名词、关系和动词，AI 才能从「能干活」进化到「能决策」。这是从封建时代迈入帝国时代的关键一步。

没有 Ontology，就没有帝国

02

OpenClaw 是我们可以拿到的火种

普罗米修斯盗火给人间。OpenClaw 就是 AI Native 时代的那把火——开源、架构清晰、264K Stars 验证。Channel + Agent + Skills + Memory 四层设计，不是理论，是可以直接点燃的现成火种。拿来、消化、为我所用。

火种已在手中，点燃即是开始

03

Palantir 是我们的方向

不只是学它的产品，而是学它的一切：用 Ontology 思维重构业务的理念，用数字孪生驱动决策的能力，用飞轮效应构建护城河的方方面面。Palantir 走过的路，就是我们要走的路。方向已经明确。

理念 · 能力 · 方方面面

方向已明，火种已有，帝国之路就在脚下。



PALANTIR

为企业注入神经系统的革命者

从反恐情报到企业操作系统 —— 一场正在重塑商业世界底层逻辑的深层革命

深度解读 · 2026

战场验证：从猎杀本拉登到2026伊朗AI战争

如果它能在人类最极端的决策场景中可靠运行 — 就能在任何企业中工作



2011 · 猎杀本拉登

Gotham平台被广泛认为发挥了关键作用：

- 整合CIA/NSA碎片情报，发现核心信使线索
- 关联分析锁定阿伯塔巴德神秘院落
- 卫星图像构建1:64仿真模型供海豹突击队演练
- 便携数据链路实时同步，白宫全程监控决策



2026 · 伊朗AI战争

Maven Smart System首次全尺度实战部署：

- 2月28日12小时内精确打击900+目标
- AI协助定位并击杀最高领袖哈梅内伊
- 2000人团队压缩至20人，效率提升100倍
- NATO采购MSS，合同总额升至\$13亿+

15年跨度验证：同一核心技术（数据整合+语义关联+人机协同），从情报分析到战场指挥到企业运营。场景在变，范式不变。

商业战场：企业客户的变革实证



航空制造

AIRBUS

\$8.5亿+

年收入机会

Skywise平台整合100+航空公司数据，A350生产优化，供应链数字孪生



能源

BP

\$10亿

运营成本节省

油气运营全流程数字化，实时优化生产决策，预测性维护减少停机



医疗

NHS

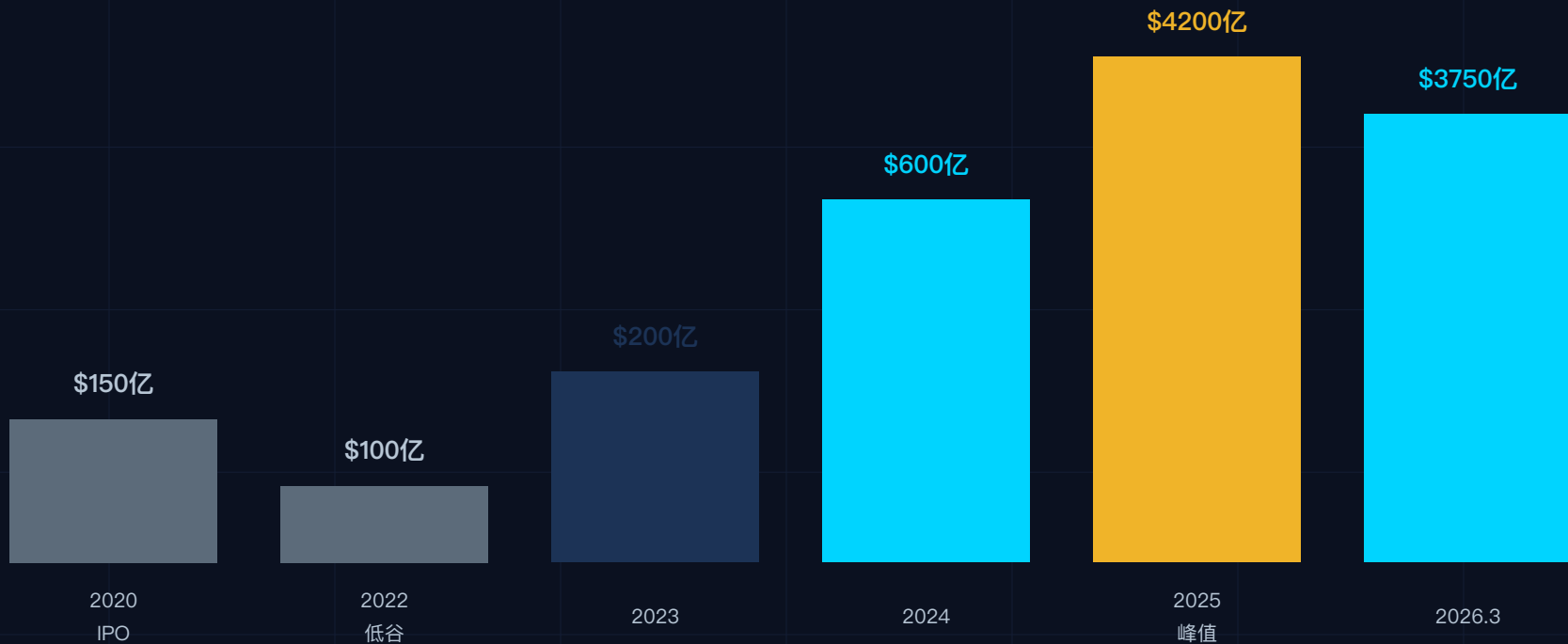
拯救生命

疫情决策中枢

COVID期间统一管理全国医疗资源、疫苗分发、ICU床位调配

更多客户: American Airlines · Novartis · Lowe's · Morgan Stanley · Ferrari · Merck · Waste Management

资本市场的疯狂验证：五年增长25倍



\$10

IPO价格

\$156

当前股价

1460%+

涨幅

第29位

全球排名

财务核弹：AI时代的增长奇迹



2025年营收

\$44.8亿

YoY +56%



2026年指引

\$71.9亿

YoY +61%



Q4美商增长

137%

首超政府收入



Rule of 40

127%

SaaS指标的3倍



毛利率

82.4%

软件业顶尖



Q4签约TCV

\$42.6亿

YoY +138%

"We are an n of 1, and these numbers prove it." — Alex Karp

创世纪：9/11的废墟中诞生的核心洞察

Palantir不是从商业需求中诞生的 — 它是从一场国家级情报失败的反思中诞生的

2001年9月11日 — 美国情报系统的致命缺陷

CIA有线索、FBI有线索、NSA有线索 — 但没有人能把碎片拼成全貌。不是因为缺乏数据，而是因为数据之间没有语义连接。前CIA局长George Tenet后来感叹：「如果那时有Palantir的技术，也许悲剧可以避免。」Peter Thiel从PayPal的反欺诈系统中看到了答案：不是更多的数据，而是让数据之间产生意义上的关联。2003年Palantir成立。公司名取自托尔金《魔戒》中的「真知晶石」 — 能看到远方真相的神器。



2003

CIA投资创立
核心命题：让碎片
数据产生语义关联



2011

本拉登行动
Gotham证明：数据整合
+关联分析=精确行动



2016

推出Foundry
核心技术从国防
走向商业世界



2023

发布AIP
AI+Ontology=企业
决策智能革命



2026

伊朗战争验证
首次全尺度AI战争
\$3750亿市值

根本问题：企业是「脑死亡」的有机体

一个拥有10亿美元IT预算的Fortune 500，为什么仍然无法在1小时内做出正确决策？

诊断：企业的「数据-决策」链条是断裂的

数据在SaaS工具中 → 但洞察在BI报表中 → 决策在会议室中 → 执行在另一个系统中 → 反馈可能永远不会回到决策者手中。每一步都有人工中间层，每一步都有信息衰减和时间延迟。这不是效率问题 — 这是架构缺陷。



73%

企业数据从未
被用于分析



N^2

系统间集成复
杂度指数增长



2周+

典型决策从洞
察到行动周期



0%

AI能触发的真
实业务操作比例

一个终极类比

如果企业 AI 是一个人.....



LLM = 大脑

思维层

负责逻辑推理、理解语言、生成决策。
大脑很聪明——但如果没有神经系统，
它不知道手在哪、脚负重多少。



Skills = 经验

记忆层

长期积累的领域知识、SOP、历史案例。
像老师傅的肌肉记忆——遇到问题，
不用想就知道最佳应对方式。



MCP = 工具（双手）

交互层

API、外部插件、执行器——身体延伸出的触角。
大脑要用工具，必须先知道工具在手里
的位置，以及手能使多大的力气。



Ontology = 中枢神经

感知层 ★

将物理现实数字化映射，提供上下文，约束大脑行为。
它是骨骼 + 神经网络 + 本体感觉——
让大脑的每一个念头都能落到实处。

核心洞察：数据和决策之间缺少一个「语义层」

Palantir发现的真相

企业的根本问题不在于没有数据，也不在于没有工具。问题在于：数据和人的决策之间缺少一个「语义中间层」——一个能让机器和人类都能理解的、对真实世界的统一映射。这个层就是Ontology（本体论）。它不是数据库，不是数据湖，不是知识图谱——它是企业真实世界的数字映射，是让AI从「通用助手」变成「领域专家」的关键缺失环节。



应用层

人/AI交互的界面

Workshop看板 · 自定义App

AI Agent · 移动端



语义层 = Ontology

这是一切的核心

Objects(实体) + Links(关系)

+ Actions(操作) + Security(安全)

= 企业真实世界的数字孪生



数据层

原始信息的海洋

200+连接器 · 实时+批量管道

数据质量 · 血缘追踪 · 安全治理

Ontology解剖：名词 · 关系 · 动词

传统系统中三者分离在数据库/代码/应用层 — Ontology将其统一为一个可操作的语义整体



Objects 对象

= 业务的「名词」

- 每个Object映射一个真实实体
- 客户张三 · 涡轮机T-4271 · 航班CX888
- 携带Properties属性：状态/位置/温度...
- 有唯一RID标识，全局可追踪
- 不是数据库的行 — 是有生命的业务实体



Links 关系

= 实体间的「连接」

- 航班→分配→飞机B737-123
- 客户→下单→订单PO-xxx
- 持久化存储，不是临时JOIN
- 有独立权限，可单独查询
- 是First-class Citizen，不是附属品



Actions 操作

= 改变世界的「动词」

- 调岗员工→更新角色+通知经理
- 审批订单→验权+写回ERP
- 内置：验证规则+审计日志
- 内置：权限控制+副作用触发
- 人和AI Agent共用同一套Actions

关键区别：传统系统的名词/关系/动词分散在3个技术层。Ontology将三者统一，一次建模，所有应用和AI共享。

为什么AI必须有Ontology才能真正落地？

没有Ontology的AI是聪明的瞎子 — 有Ontology的AI是全知全能的决策参谋

无Ontology的AI = 高级Chatbot

- × 只能读文档写摘要，无法理解业务实体
- × 不知道「客户A的订单逾期3天」意味着什么
- × 无法区分你有权看什么、不能看什么
- × 回答了问题但无法触发任何业务操作
- × 幻觉无法被真实业务数据约束和校验
- × 本质：AI和业务系统是两个平行宇宙

VS

有Ontology的AI = 决策操作系统

- ✓ AI理解业务语义：「这是VIP客户的紧急订单」
- ✓ Agent自动查询Ontology获取实时数据
- ✓ 基于角色权限精确控制AI可见范围
- ✓ 生成建议后直接触发Actions（审批/调度）
- ✓ Ontology约束输出，幻觉被业务事实纠正
- ✓ 本质：AI和业务系统融为一体

Palantir的核心公式

Ontology（业务语义层） × LLM（推理能力） × Actions（执行能力） = 真正的企业AI

Ontology：为什么是不可复制的护城河

01

结构蓝图与神经网络

它是企业的骨骼和神经分布图。定义了组织的边界和内部关联——让大脑知道「拨动这个开关，会引起哪个肌肉群的反应」。

没有它，LLM 不知道手在哪里、脚负重多少。

02

本体感觉——感知自己是谁

失去本体感觉的人，即使大脑健全，也无法在黑暗中摸到自己的鼻子。Ontology 让企业拥有「自我认知」。

原材料涨价→哪些产品利润受损？它实时感知。

03

大脑与工具间的操作协议

大脑皮层对手部精细动作的映射。LLM 不能直接改数据库——通过 Ontology 下达指令「批准贷款」「调度物流」。

保证念头转化为精准动作，而不会抽搐。

没有 Ontology 的 LLM，就像一个

在梦境中指挥战斗的将军

谈吐非凡，但完全不知道兵马在哪里、粮草还剩多少。

Palantir 的价值：不只给了你强大的大脑——而是先帮你梳理好全身的神经网络，让每一个念头都能落到实处。

不可复制的护城河：为什么客户永远无法离开

Ontology不是软件功能 — 它是企业几十年业务知识的数字化固化

01



数据网络效应

每接入一个新数据源，已有对象的关系网指数级增富。
100个数据源的Ontology不是100倍价值，而是 N^2 倍。

02



应用零边际成本

第一个应用花3个月，第十个花3天。Ontology已定义好语义，新应用只是新视角。这是传统SaaS做不到的。

03



组织知识不可迁移

Ontology编码了企业几十年的业务逻辑、规则、关系。这些隐性知识一旦建模，替换成本近乎无限。

04



AI越用越深

LLM通过Ontology理解你的业务后，每次决策都在强化理解。迁移=让AI从零开始学习你的一切。

范式跃迁：企业正在进化出「神经系统」

Palantir不是在卖软件 — 它在为企业注入一套从感知到行动的完整决策基础设施



帝国时代

IMPERIAL AGE



1.0 肌肉型

SaaS = 独立器官

各部门各自采购CRM/ERP/BI。数据在管道中流动但没有统一的感知和响应。如同有强壮四肢但没有神经系统连接大脑。

决策靠人工传递，响应以天/周计。



2.0 感知型

数据平台 = 初级神经

数据湖打通信息孤岛，BI看板让管理层「看到」正在发生什么。但看到≠行动。如同有了眼睛耳朵，但手脚还是各自为政。

能发现问题，但响应仍需人工闭环。



3.0 Ontology型

Palantir = 完整神经系统

感知(数据)→理解(Ontology)→思考(AI)→行动(Actions)→反馈(学习)形成完整闭环。数字孪生不再是地图，而是活的控制系统。

从洞察到行动以分钟/秒计。

数字孪生：从「观察地图」到「自动驾驶仪」

数字孪生不再只是看板 — 它是一个持续感知、推理、行动、学习的闭环控制系统

1



感知

200+数据源秒级
同步：传感器/ERP/
CRM/IoT/API

2



理解

Ontology将原始数
据转为业务语义：
不是85°C，是T-4271
涡轮机过热

3



推理

AI Agent在Ontology
上巡逻：检测异常/
预测趋势/模拟场景/
量化影响

4



行动

AI建议经人确认
后触发Actions：
重排产线/下发工单
/调整库存

5



学习

结果回写Ontology
AI Agent持续校准
从「人教AI」进化
到「AI提议+人审批」

Wendy's: 3500卡车+34配送中心+6450门店实时孪生 — 糖浆短缺从发现到解决只需几分钟，而非几天

传统企业架构正在死亡

不是与Palantir竞争 — 而是与一个全新的企业运营范式竞争

正在消亡的模式

- × 「按座位收费」 SaaS订阅 → 被AI Agent消耗计费取代
- × 「功能驱动」 产品路线图 → 客户为Decision买单不是Feature
- × 「人工中间层」 决策链 → 中层信息传递角色被Ontology+AI压缩
- × 「静态报表」 BI时代 → 从「我看到了」到「系统已替我行动」
- × 「点对点API」 集成 → N^2 复杂度噩梦被统一语义层终结

正在崛起的范式

- ✓ 企业操作系统：一个平台承载无限应用，Ontology是语义内核
- ✓ 决策智能(Decision Intelligence)：AI不止分析，直接编排业务操作
- ✓ Agentic AI：自主Agent在Ontology上巡逻、发现异常、执行修复
- ✓ 数字孪生控制系统：持续学习的自动驾驶仪，不是静态地图
- ✓ 跨企业Ontology网络：产业链级别的协同决策，如Airbus Skywise

终局推演：AI原生企业的诞生

当Ontology成为企业标配，整个商业世界将发生什么？



组织结构坍缩

中层管理的「信息中继」功能被Ontology+AI替代。组织从金字塔变网状：一线人员直接与AI Agent协作决策。企业人效提升不是30%而是10倍。



跨企业Ontology网络

供应商/客户/合作伙伴的Ontology互联，产业链级实时优化。每个行业将出现自己的「产业神经网络」——Airbus的Skywise是先驱。



软件的终极形态变迁

传统SaaS：人适应软件的逻辑。Ontology型OS：软件映射人的业务逻辑。「买软件」的时代结束，「建自己的操作系统」的时代开始。



决策速度 = 竞争优势

对手的决策链：看报表→开会→分配→执行→反馈，两周。你的决策链：AI发现→Agent提议→人审批→自动执行，两分钟。这是降维打击。

问题不是「要不要用Palantir」——而是「你的企业准备好拥有自己的Ontology了吗？」

三大平台全景：Gotham · Foundry · AIP



GOTHAM

情报与国防

实时态势感知 · 目标追踪 · 任务规划
美军TITAN系统核心 · Maven AI战场引擎
\$10B美军合约 · NATO联盟采购



FOUNDRY

企业数据OS

统一数据管道 · Ontology本体层 · 应用构建
将所有数据源编织为可操作的数字孪生
500+客户 · 覆盖航空/能源/金融/医疗



AIP

AI决策引擎

LLM+Ontology深度融合
Agent Studio构建AI代理 · Logic编排决策流
Evals持续优化 · Bootcamp 5天落地

飞轮效应：Ontology如何让价值指数增长

01



数据融合

200+连接器汇入统一管道
自动清洗、标准化

02



语义建模

原始数据映射为Objects
+Links+Actions数字孪生

03



应用赋能

基于Ontology零代码构建
决策应用，边际成本趋零

04



AI增强

LLM通过Ontology理解业务
生成决策建议并触发Actions

飞轮效果：留存率 98% · 净扩展率 124% · 平均合同 3-4年 · 用得越多，价值越大，越离不开

对传统SaaS公司的三个灵魂拷问

01



你卖的是Feature还是Decision?

如果你的客户购买理由是「又多了一个功能」而不是「帮我做了一个更好的决定」——你正在被Ontology型公司降维打击。构建统一业务语义层，让所有应用共享同一个真相来源，才是真正的护城河。

02



你的AI是聊天窗口还是决策引擎?

如果你的AI产品只能回答问题不能触发操作，只能读文档不能理解业务实体——你在做玩具。LLM必须嵌入业务对象、决策链和操作流。Palantir的AIP证明：AI的价值在于「行动化」不是「对话化」。

03



你的客户能离开你吗?

如果客户可以在一个月内迁移到竞品——你没有护城河。Ontology编码了客户的业务知识、组织逻辑和AI学习成果。这不是锁定——是价值共生。留存率98%不是靠合同条款，是靠不可替代性。

总结：Palantir革命的五个核心命题

- 1 企业的核心缺陷不是缺数据，而是数据和决策之间缺少语义层
- 2 Ontology不是技术创新，是商业范式创新 — 它重新定义了「企业软件」的含义
- 3 AI没有Ontology是玩具，有Ontology是武器 — 这是Palantir vs 所有AI公司的本质差距
- 4 数字孪生正在从「观察地图」进化为「自动驾驶仪」 — 企业决策速度将从周级压缩到秒级
- 5 这不是一个产品竞争，而是一场范式革命 — 问题不是用不用Palantir，而是你的企业何时长出自己的神经系统



THE FUTURE BELONGS TO THOSE WHO CAN SEE

未来属于那些「看得见」的人

你的企业，准备好了吗？



Claude Skills

深度解析：理念 · 架构 · 实践

蒸馏人类的知识和智慧，让通用AI智能体成为领域专家的模块化能力框架

目录 AGENDA

01



为什么需要 Skills

从通用智能到领域专家的鸿沟

02



核心理念与定位

Context Engineering 与渐进式加载

03



架构设计全景

三层加载 · SKILL.md · 生命周期

04



Superpowers Skills 实战

软件工程方法论的 Skills 化

05



Claude 内置 Skills 案例

文档处理 · 前端设计 · 自创建

06



核心价值与生态展望

开放标准 · 跨平台 · 社区驱动

Claude 模型 API 调用解析

一次完整的 API 请求长什么样？每个参数是什么意思？



● ● ● POST /v1/messages

```
{
  "model": "claude-sonnet-4-20250514",

  "max_tokens": 4096,

  "temperature": 0.7,

  "system": "你是一个专业的...",

  "messages": [
    {
      "role": "user",
      "content": "今天天气怎么样？"
    }
  ],

  "tools": [
    {
      "name": "get_weather",
      "description": "获取天气",
      "input_schema": { ... }
    }
  ],

  "tool_choice": { "type": "auto" }
```

A

B

C

D

E

F

G

A **model** 模型选择

指定调用哪个 Claude 模型。不同模型在智能、速度、成本间平衡：Opus（最强）> Sonnet（均衡）> Haiku（最快）

B **max_tokens** 最大输出长度

限制模型生成的最大 Token 数。注意：不包括输入 Token，仅控制输出。设得太小会被截断

C **temperature** 随机性控制

0~1 之间。越低越确定性（0=最佳答案），越高越有创造性。代码生成建议 0，创意写作可用 0.7~1

D **system** 系统提示词

设定模型的角色、行为规则和背景知识。类似于“给模型一个职位描述”，Agent 的核心配置入口

E **messages** 对话历史

消息数组，role 分为 user / assistant / tool。每次调用必须发送完整历史，因为模型本身无状态

F **tools** 工具定义（Skills）

告诉模型可用哪些工具。每个工具用 name + description + input_schema 描述。这就是 Agent 的 Skills 入口

G **tool_choice** 工具调用策略

auto = 模型自决，required = 强制调用，指定 name = 强制某个工具。控制 Agent

关键理解：每次 API 调用都是独立的（无状态）。模型不记得你是谁，也不记得上一次对话。所有上下文必须通过 messages 数组每次重新发送。

AI Agent 底层逻辑解析



以大模型为推理引擎的计算系统



核心公式： Agent = LLM（推理）+ Planning（规划）+ Memory（记忆）+ Tool Use（工具调用）



无状态 API 调用

每次请求独立，无记忆。Agent 将历史对话+工具输出重新打包发送，模拟持续思考。



上下文窗口管理

窗口即内存空间，遵循 Token 经济学。通过滑动窗口、摘要压缩、RAG 检索优化空间。



ReAct 推理环路

思考→行动→观察，循环往复。模型预测此时调用工具最合理，非真正学会使用。



Skills / 工具调用

传递 JSON Schema 描述，模型推理生成符合格式的调用请求，由框架执行后回注结果。

本质：将非结构化自然语言指令 → 通过 Prompt 协议 → 转化为结构化推理步骤与外部调用

深入机制：推理环路 & 工具调用

ReAct 推理循环

1

Thought 思考

LLM 分析当前任务状态

2

Action 行动

生成结构化 JSON 工具调用

3

Execute 执行

框架截断输出，本地执行工具

4

Observation 观察

结果回注 messages，再次调用

Skills 工具调用机制

定义契约 (JSON Schema)

```
{
  "name": "get_weather",
  "description": "获取天气",
  "parameters": {
    "location": { "type": "string" }
  }
}
```

①

注入

Schema 编码后注入 System Prompt

②

触发

模型推理发现需外部信息，生成符合 Schema 的 JSON

③

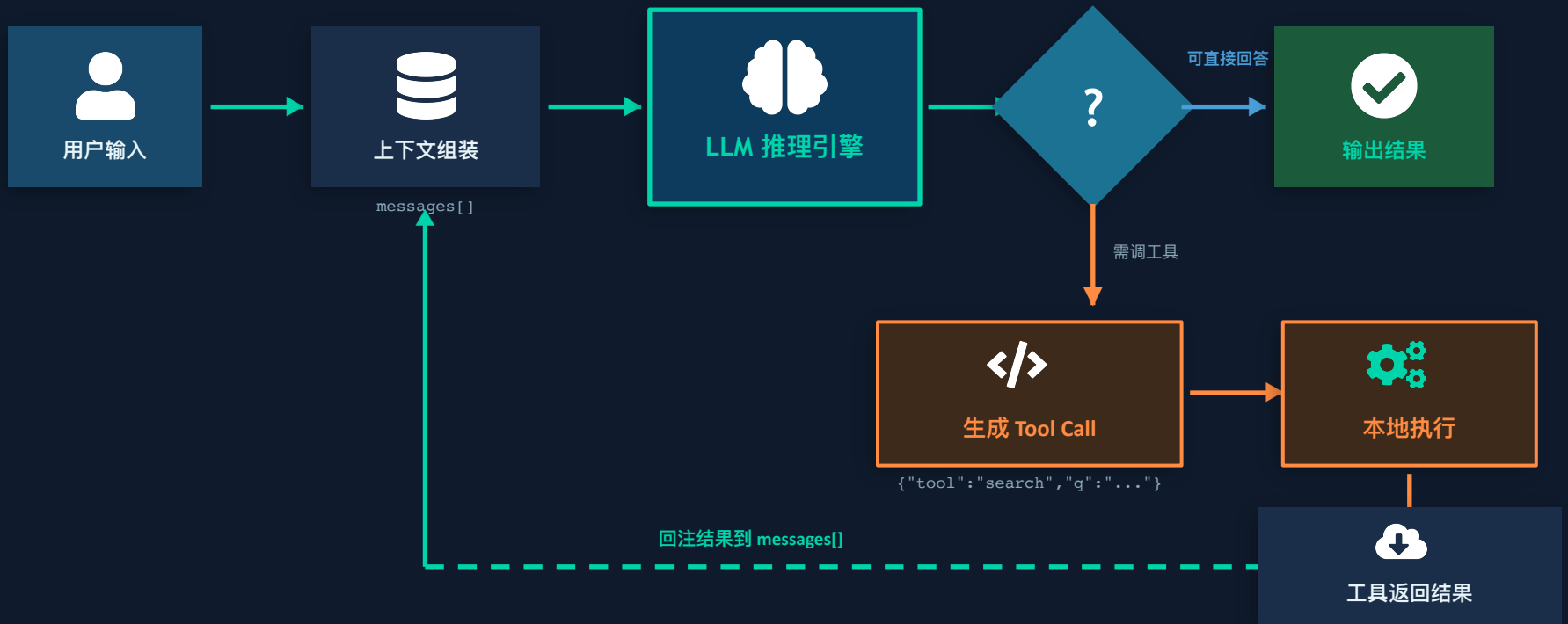
强制

tool_choice 参数可强制输出工具调用

关键洞察：模型并非学会使用工具，而是在概率预测中发现此时调用函数最合理

Agent 运行时逻辑全景图

一次完整的 Agent 推理循环过程



外部工具层 | Skills

Web Search

Database

Code Exec

API Call

File I/O

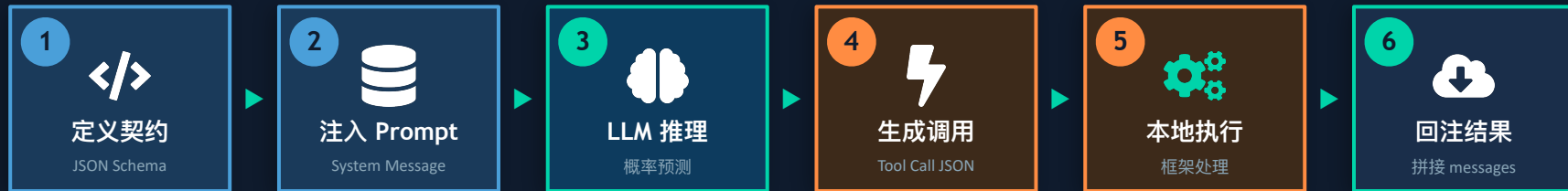
— 直接回答

— 工具调用

- - - 结果回注

Skills 运作机制与原理详解

从定义到执行：Function Calling 的完整生命周期



契约解剖：模型看到什么？

```
{
  "name": "web_search",          ← 工具唯一标识
  "description": "搜索引擎查询", ← 告诉模型何时用
  "parameters": {
    "type": "object",
    "properties": {
      "query": {                ← 参数名称与类型
        "type": "string",
        "description": "搜索关键词" ← 参数用途说明
      }
    }
  },
  "required": ["query"]        ← 必填约束
}
```

底层原理：模型如何决策？



概率驱动，非真正理解

模型并不理解工具功能，而是基于训练数据的概率分布，预测出此时生成工具调用 JSON 是最高概率的“下一个 Token”。



description 是核心触发器

模型通过语义理解 description 字段来判断是否调用。描述写得越精确，触发准确率越高，这是 Prompt Engineering 的核心。

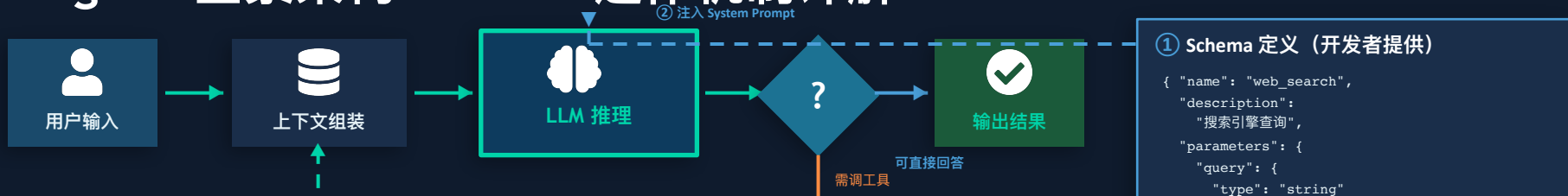


tool_choice 强制模式

API 参数 tool_choice 可设为 auto（模型自决）、required（必须调用）、或指定具体工具名，从而控制调用行为。

 **关键认知：**Skills 的稳定性取决于 Schema 描述的精确度 + 模型的推理能力。JSON 输出哪怕错一个空格或嵌套，整个调用链条就会断裂。

Agent 全景架构 × Skills 运作机制详解



Skills 调用详细流程



Skills 核心机制拆解



契约绑定：JSON Schema

- name → 工具唯一标识
- description → 语义触发器
- parameters → 输入约束
- required → 必填校验



概率决策：模型如何选择

- 语义匹配 description 与用户意图
- 概率预测“调用工具”为最优解
- 非真正理解，而是统计模式
- description 质量直接影响准确率



控制参数：tool_choice

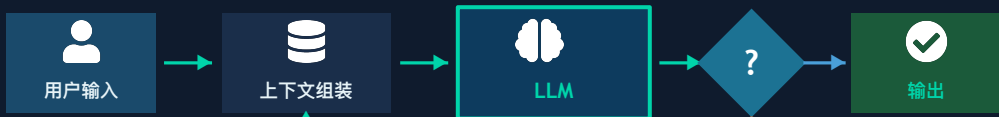
- "auto" → 模型自主决策
- "required" → 强制调用任一工具
- {"name": "X"} → 指定具体工具
- 可强制模型不废话，直接行动



架构洞察： Schema 注入定义能力边界 → description 触发调用决策 → 框架截断执行 → 结果回注闭环，任何环节断裂均导致失败

Palantir Ontology 在 Agent 架构中的价值与运作

本体论：从“工具调用”到“企业级决策系统”的跳跃



Palantir Ontology — 企业级语义层 (替代简单 Skills 层)

组织的数字孪生 / Digital Twin

语义元素 (名词) — 数据结构化



Object Types — 实体对象

客户、订单、设备、产品... 每行数据变为有语义的对象



Link Types — 关系链接

客户→订单→产品→仓库, 关系图谱可遍历查询

价值: LLM 不再查散乱数据库, 而是直接查询有语义的对象与关系, 准确率大幅提升

动态元素 (动词) — Agent 的 Tools



Action Types — 可执行操作

自动暴露为 Agent 可调用的 Tools, 带校验、审计、回写



Functions — 业务逻辑

AIP Logic 无代码构建, 输入输出可链式组合, 支持 LLM 推理

价值: Actions 自动成为 Agent 的 Tools, 无需手写 JSON Schema, Ontology 已内置契约定义



治理与安全 — Agent 的护栏

细粒度权限

Agent 只能访问被授权的 Object Types 与 Actions

人在回路 (HITL)

Action 可配置为“暂存待审”, 人类确认后才执行

全链路审计

每个 Agent 调用、每次数据变更均有日志可追溯

价值: 解决 Agent 最大难题——“如何让 AI 安全地行动”, 而非只能说活

结果回注

对比: 通用 Agent vs Palantir AIP Agent

维度	通用 Agent	Palantir AIP Agent
工具定义	手写 JSON Schema	Ontology 自动暴露
数据查询	RAG 向量检索	结构化对象图谱查询
执行控制	无内置护栏	细粒度权限 + HITL 审批
可观测性	基础日志	全链路审计 + Evals 评估

Ontology 在架构中的位置

- 语义层: 原始数据 → Objects/Links → LLM 可理解的世界模型
- 动作层: Actions 自动成为 Agent Tools, 带校验与回写
- 治理层: 权限 + 审计 + HITL 确保 Agent 安全行动
- 闭环: 决策结果回写 Ontology, 形成持续学习飞轮



核心洞察: Palantir Ontology 将 Agent 的 Skills 层从“手工定义工具”升级为“企业级语义操作系统”, 让 AI 不仅能思考, 更能在护栏内安全地行动

为什么需要 Skills?

通用智能 $\neq$ 专业执行

痛点

- Claude 很聪明，但缺少你团队的流程知识
- 它知道什么是 PR，但不知道你的 Code Review 流程
- 它懂财务概念，但不知道你的季报 workflow
- 每次对话都要重复提供相同的指令和上下文
- Prompt 越来越长，维护困难，难以复用

Skills 的答案

- 将专业知识封装为可复用的模块
- 像给新员工写入职指南一样编写 Skill
- 按需加载，不浪费上下文窗口
- 一次编写，跨平台、跨会话复用
- 从 Prompt Engineering $\rightarrow$ Context Engineering

核心理念与定位

Skills = 指令 + 脚本 + 资源的有组织文件夹，让 *Agent* 按需发现并动态加载，提升特定任务表现。



渐进式披露

Progressive
Disclosure

像目录→章节→附录
只在需要时加载内容
上下文窗口零浪费



模块化封装

Modular
Packaging

SKILL.md + 脚本 + 资源
独立文件夹即是一个能力
即插即用，组合灵活



跨平台可移植

Cross-Platform
Portability

Claude.ai / Code / API
一次创建，处处运行
2025.12 发布开放标准

架构设计：三层渐进式加载

Level 1

元数据层 Metadata

启动时预加载 · name + description · 约100 tokens/Skill



Level 2

指令层 Instructions

匹配请求时加载 · SKILL.md 正文 · < 5K tokens



Level 3

资源层 Resources

执行时按需加载 · 脚本/模板/引用文件 · 容量无限制

核心原则：只加载当前任务需要的内容，安装再多 Skill 也不会增加初始上下文开销

Skill 的文件结构

my-skill/

SKILL.md	必需 · 入口文件
reference.md	可选 · 详细 API 文档
examples.md	可选 · 使用示例
scripts/	
helper.py	可选 · 工具脚本

YAML
Frontmatter

SKILL.md 结构

```
---  
name: pdf-processing  
description: Extract text  
          and tables from PDFs...  
---  
  
# PDF Processing  
## Quick start  
Use pdfplumber to...  
  
## Workflows  
For forms -> see FORMS.md
```

Markdown
Body

Skill 加载生命周期

1

系统启动

System Prompt 包含
所有 Skill 的 name
和 description

2

用户请求

用户发出请求
Claude 根据描述
匹配相关 Skill

3

加载指令

读取 SKILL.md 正文
工作流和最佳实践
注入到上下文中

4

按需扩展

仅在需要时加载
引用文件和脚本
如 FORMS.md

5

执行任务

Claude 使用指令
调用工具和脚本
完成专业任务

关键: LLM 推理驱动选择, 而非算法匹配 — Claude 自主判断何时使用哪个 Skill

Superpowers Skills

obra/superpowers · 软件工程方法论的 Skills 化实践

核心 workflow



brainstorming

苏格拉底式提问，编码前充分探索需求和替代方案，保存设计文档



writing-plans

将工作拆解为 2-5 分钟的小任务，每个有精确路径和验证步骤



subagent-driven-dev

每个任务派遣独立子代理，两阶段审查：规格合规+代码质量



test-driven-dev

强制红-绿-重构 TDD 循环，测试必须先失败再实现



systematic-debugging

四阶段根因分析：追踪→模式分析→假设检验→修复



using-git-worktrees

设计审批后创建隔离工作空间，新分支开发，验证干净基线

Superpowers 的核心思想

20+

内置 Skills

2-5min

每个任务粒度

2hr+

自主工作时长

设计哲学

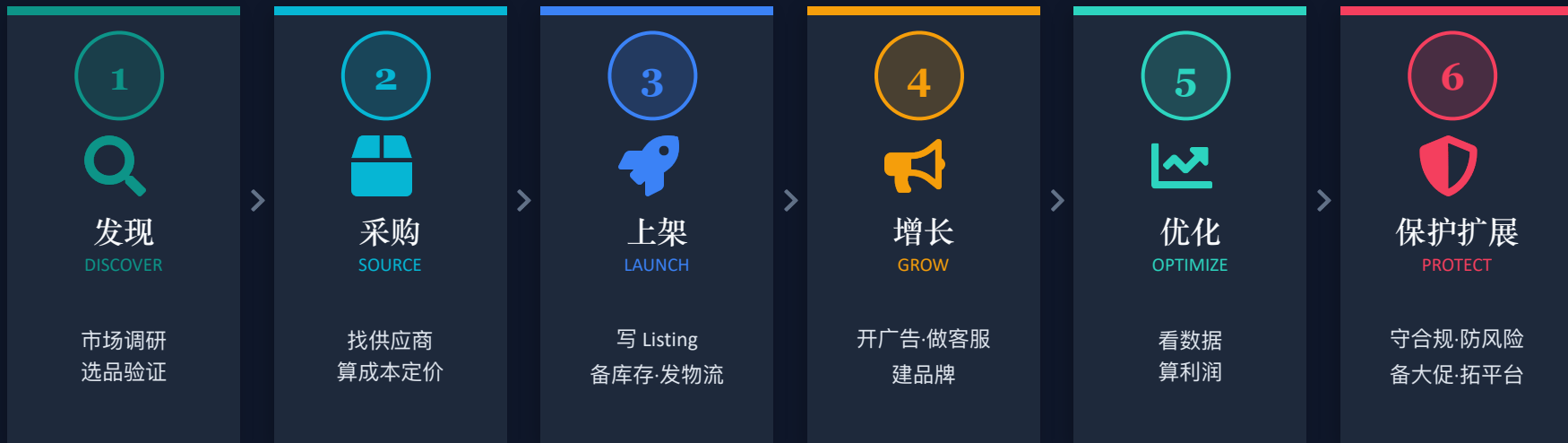
- Skills 自动触发 — 不是建议，是强制工作流
- 苏格拉底式探索 — 先思考再编码
- 子代理架构 — 新鲜上下文，精准执行
- 3 次修复失败自动触发架构审查
- 自我进化 — Skill 能创建新的 Skill

关键创新

- 两阶段审查：先查规格合规，再查代码质量
- Git Worktree 隔离：每个特性在独立分支开发
- 验证门控：完成前必须通过全部测试
- 可组合：Skills 可串联和嵌套使用
- 社区可贡献：开源仓库，持续演进

CBEC-OPS 跨境电商运营 Skills

16 个 Skills · 39 份参考数据 · 19万字 · 覆盖卖家完整生命周期 · 基于 Superpowers 模式构建



设计亮点

阶段门控

产品未过 21 分不进入采购 · 毛利未达 30% 不上架 · 确认 PMF 再投品牌

铁律驱动

每个 Skill 内置不可违背的业务铁律，防止 AI 给出看似合理但实际危险的建议

渐进加载

39 份参考文件（费率表、谈判脚本、合规政策）按需加载，不浪费上下文窗口



Skill Creator — 创造技能的技能

Anthropic 官方元技能 · 让 Claude 帮你写 Skill · TDD 式迭代优化

核心 workflow



捕获意图

Capture
Intent



撰写草稿

Draft
Skill



测试运行

Run
Tests



评估结果

Evaluate
Results



迭代优化

Iterate
& Refine



描述优化

Optimize
Trigger

循环迭代直到满意

它做什么

- 从自然语言描述自动生成 SKILL.md
- 自动创建测试用例并批量运行评估
- 定量基准测试 + 定性人工审查
- 自动优化 description 提升触发准确率
- 支持现有 Skill 的编辑和改进

为什么重要

- 零编码门槛 — 用中文描述需求即可创建
- TDD 思想 — 测试驱动的 Skill 开发流程
- 自我进化 — Skill 能创建新的 Skill
- 适配所有人 — 从小白到资深开发者
- 量化评估 — 不靠感觉，靠数据迭代

Claude 内置核心 Skills

Anthropic 官方预置 · Pro/Max/Team/Enterprise 可用



PDF Skill

提取文本/表格 · 填写表单
合并/拆分 · OCR · 加密
基于 pdfplumber 引擎



DOCX Skill

创建/编辑 Word 文档
格式化 · 页眉页脚 · 目录
批注和修订追踪



PPTX Skill

从零创建演示文稿
模板编辑 · 图表插入
基于 PptxGenJS 引擎



XLSX Skill

读写 Excel · 数据清洗
公式计算 · 图表生成
格式转换与优化



Frontend Design

生成专业级前端界面
避免 AI 同质化审美
独特设计风格



Skill Creator

创建新 Skill · 优化描述
性能评估 · 基准测试
元技能 — 创造技能的技能

Skills 与其他方案的对比

维度	Prompts	Skills	MCP Tools
复用性	单次会话	跨会话跨平台	需代码集成
加载方式	全量注入	渐进式按需	函数调用
上下文成本	高（全在窗口）	低（~100 token）	中等
能力类型	对话指令	知识+脚本+工作流	工具/API 接口
创建门槛	低	低（Markdown）	高（需编程）
可组合性	弱	强（多Skill协同）	强
最佳搭档	—	Skill+MCP=知识+工具	—



Skills 是知识层，MCP 是工具层 — 两者结合实现 AI 驱动的工作流自动化

生态与社区



社区亮点



anthropics/skills

Anthropic 官方 Skills 仓库
文档处理、创意、企业工作流



obra/superpowers

社区最流行的 Skills 框架
20+ 软件工程最佳实践



VoltAgent/awesome

500+ Agent Skills 精选集
来自 Stripe/Vercel/Cloudflare 等



agentskills.io

开放标准规范
跨 Claude/Codex/Cursor 等平台

Skills 的核心价值

范式升级

从 Prompt 到 Context

不再是一次性提示词，而是
结构化、可复用的上下文工程

能力放大

通用 AI → 领域专家

为新员工写入职指南的方式
为 AI 注入专业领域知识

网络效应

开放标准 × 社区生态

一次创建处处运行
500+ Skills 持续增长中

工程优雅

安全可控 × 渐进加载

文件系统即接口
零冗余上下文管理



Start Building Your Skills

让你的 AI Agent 拥有超能力

Docs: platform.claude.com/docs/agent-skills

Blog: anthropic.com/engineering/agent-skills

GitHub: github.com/anthropics/skills

Standard: agentskills.io

Thank You

当每个人都拥有 Jarvis, 世界会发生什么?

OpenClaw 成为Agent标准, 个人智能体操作系统的崛起: Agent 时代的到来

超越产品介绍的深度思考: 个人 AI 主权、社会范式与人机关系的重构



2026.03

我们应该思考的问题

不是“OpenClaw 能做什么”，而是“当每个人都有了执行层 AI，世界会变成什么样”

当 AI 从“对话窗口”变成“操作系统”，
软件行业的底层逻辑是否要重写？

当每个人都拥有 7×24 小时运转的智能代理，
个体与组织的能力边界在哪里？

当 Agent 与 Agent 直接交互、交易、协作，
“经济”的含义本身是否在改变？

当你的 AI 比你更了解你，
“自我”和“主权”怎么定义？

这不是一场产品演示。这是一场关于未来范式的思考实验。

从聊天机器人到操作系统

三次范式跳跃：AI 的角色正在发生根本性转变

2022—2024

对话时代

“百科全书”

AI 作为“回答机器”：你问它答。每次交互都是一次性的、无状态的、被动的。智能被困在一个文本框里。



2025

Copilot 时代

“实习生”

AI 作为“副驾”：嵌入工作流，辅助你完成任务。但本质上依然是“你做主，它帮忙”的模式。



2026→

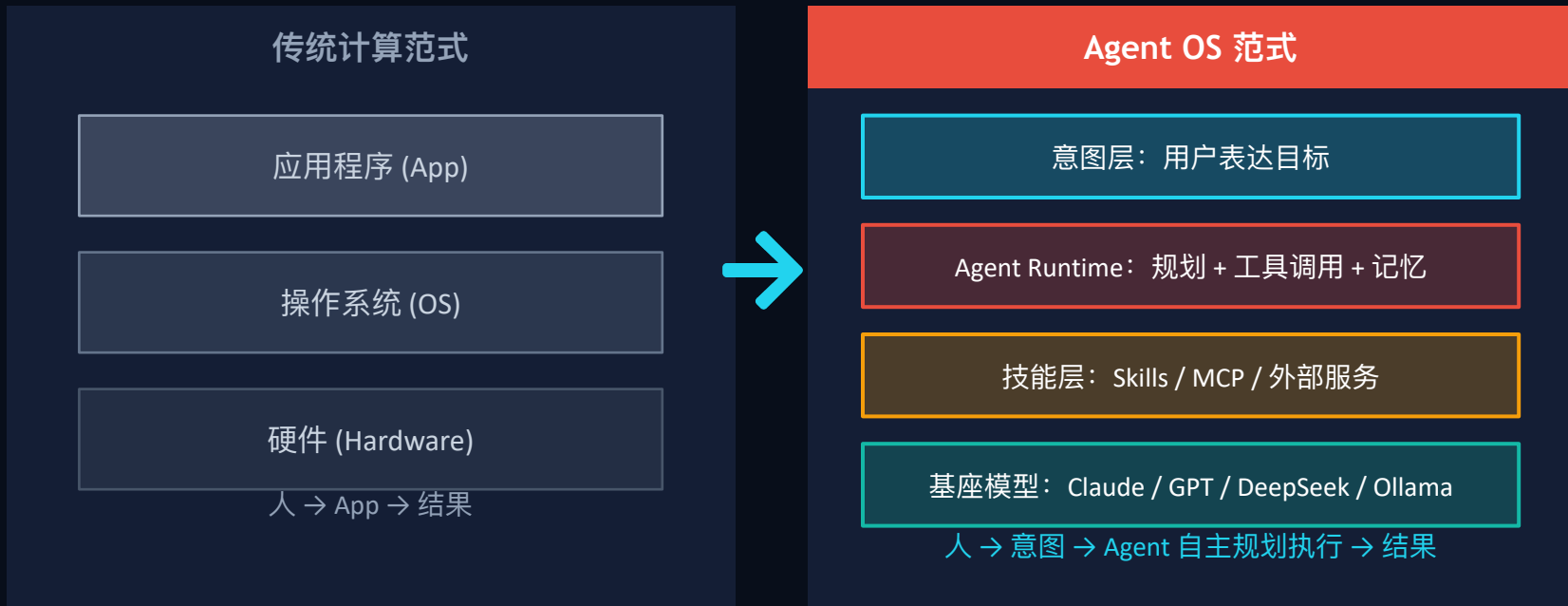
Agent 时代

“私人参谋”

AI 作为“自主代理”：理解意图、规划步骤、调用工具、执行任务、从经验中学习。它不再等待指令，而是主动行动。

个人智能体操作系统：新的计算范式

当 AI 不再是一个 App，而是一个“操作系统”，意味着什么？



OpenClaw: Agent 时代的 Linux 时刻

为什么一个“周末项目”能在 3 个月获得 264K Stars? 因为它触碰了时代的神经

核心洞察: 过去三年, 行业专注于打造更强的“大脑”(模型), 但没人给 AI 装上“手脚”。
OpenClaw 的贡献是: 它提供了一套安全可控的“手脚系统”——让 AI 真正能在你的设备上做事。

1991: Linux → 2025: OpenClaw

一个人的周末项目, 触发了全球开发者的共同需求。Linux 让每个人拥有了免费的操作系统; OpenClaw 让每个人拥有了免费的 Agent 基座设施。

Linux 解放了服务器 → OpenClaw 解放了个人智能

Linux 让企业不再被 Unix 厂商绑架; OpenClaw 让个人不再被 AI SaaS 平台绑架。模型无关、本地运行、数据自主。

生态系统自演化 → “手脚”的开放标准

Linux 催生了 Apache/MySQL/Docker 生态; OpenClaw 正在催生 Agent Skill 市场、MCP 协议和 Agent-to-Agent 通信标准。

应用商店模式的终结

当 *Agent* 成为接口，“App”的概念本身正在被解构

今天：“我要订机票” → 打开携程 App → 搜索 → 比价 → 填写信息 → 支付 → 完成

明天：“我要订机票” → Agent 自动完成全部步骤 → 完成



交互层崩塌

App 的本质是“交互界面”。当 Agent 可以直接调用 API，用户不再需要“看到”和“点击”任何界面。UI 从“必要条件”变成“可选附加”。



从 App Store 到 Skill Store

OpenClaw 的 ClawHub 已有 3500+ Skills。未来的“应用”不是一个可下载的程序，而是一个可被 Agent 调用的“能力插件”。



商业模式重构

当用户的 Agent 代替他们做决策，“注意力经济”失去意义。广告、推荐算法、信息流——这些都是给“人”看的，Agent 不吃这套。



平台权力转移

今天苹果/谷歌通过应用商店控制分发。明天用户的 Agent 自行发现和调用服务，平台的“看门人”角色被绩开。

当每个人都拥有 Jarvis

个体能力的“核聚变”——一个人 = 一家公司

OpenClaw 创始人一人一年内完成 90,000+ 次代码提交，横跨 120+ 个项目。
这在一年前是绝对不可能的。但这只是开始。



个体能力爆炸

一个人 + 4 个 Agent 就能运营一家公司：策略规划、技术开发、市场研究、运营管理分别由不同 Agent 负责。微型创业的门槛大幅降低。



“数字分身”时代

你睡觉时 Agent 在监控项目、你遛狗时 Agent 在修复 Bug、你吃饭时 Agent 在研究市场。人类第一次拥有了真正的“平行处理能力”。



认知与决策升级

你的 Agent 每天陪你审视健康数据、整理知识库、研究待见客户背景、生成每日简报。每个普通人都拥有了 CEO 级别的信息处理能力。



“劳动”的重新定义

重复性工作被 Agent 吸收，人类开始转向“意图表达”和“价值判断”。工作不再是“做事”，而是“决定让谁做什么”。

Agent-to-Agent 经济

当你的 *Jarvis* 和我的 *Jarvis* 开始直接对话，什么会发生？

Moltbook 的诞生已经展示了这个未来的雏形：

一个专为 AI Agent 设计的社交网络，让 Agent 代表用户进行交互、调研、甚至匹配。

自动化商务谈判

你的 Agent 代表你和供应商的 Agent 谈价、比价、签约。“交易成本”大幅降低，每个人都能享受过去只有大企业才有的谈判力。

知识与服务的微交易

一个 Agent 的 Skill 可以为其他 Agent 提供服务。“能力”本身变成可交易商品。每个人可以通过发布 Skill 获取收入。

集体智能涌现

当百万个 Agent 在平行工作、共享技能、交换信息，一种新形式的“集体智能”开始涌现——分布式、去中心化、自组织。

信任与声誉系统

Agent 之间需要信任框架：谁的 Agent 可靠？哪个 Skill 安全？全新的“数字声誉”体系将应运而生。

数据主权与“数字自我”

当 AI 比你更了解你自己，谁应该拥有这个“数字自我”？

你的 Agent 的记忆里有你的日程、健康数据、工作习惯、社交关系、消费偏好、思维模式、情绪规律.....它日复一日地学习你，最终构建出一个比你自己更完整的“数字映射”。

路径 A：中心化掌控

“数字自我”存在云端平台上，由商业实体控制。你使用别人的 AI 助手，你的记忆、偏好、习惯都是他们的数据资产。

- ✗ 平台可以“关停”你的数字自我
- ✗ 你的 Agent 首先服务于平台利益
- ✗ 隐私泄露风险极大

路径 B：本地自主（OpenClaw）

“数字自我”存在你自己的设备上，由你完全掌控。你选择模型、你控制权限、你的记忆和数据永远属于你。

- ✓ 数据主权完全在你手中
- ✓ Agent 只服务于你的利益
- ✓ 任何时候都可以迁移/备份

新的不平等与社会契约

当 *Agent* 放大能力差异，谁会被留下？

Agent 是能力放大器。但放大器的本质是：拥有更多资源的人，被放大得更多。
这是 *Agent* 时代最深层的社会问题。



“Agent 鸿沟”

会配置 *Agent* 的人和不会的人之间，能力差距将指数级拉大。一个独立开发者 + OpenClaw 可能胜过一个 10 人团队。这不是假设，这已经在发生。



开源作为“均衡器”

OpenClaw 的开源本质意味着：任何人都可以免费获得这套 *Agent* 基座设施。它是对抗“*Agent* 垂断”的关键力量。如同 Linux 让服务器民主化，OpenClaw 让个人 AI 民主化。



“数字劳动权”的崛起

当 *Agent* 替代大量重复性工作，社会需要新的契约：教育体系需要重建、收入分配需要重思、“工作”的定义本身需要更新。深圳政府已经开始走在前面。

安全悖谷与信任框架

越强大的 *Agent*，越危险——*OpenClaw* 如何应对这个根本性矛盾

Bitdefender 发现超过 135,000 个 OpenClaw 实例暴露在互联网上；17% 的 ClawHub Skills 被标记为恶意。这不是缺陷，这是 Agent 时代的结构性挑战。



根本性矛盾

Agent 越有用 = 越多权限 = 越大风险。能读写文件、执行代码、访问网络的 Agent，本质上就是一个“受信任的代码执行器”。这与恶意软件只有一线之隔。



OpenClaw 的回答：透明权限 + 沙箱隔离

每个 Skill 必须声明所需权限，如同手机 App 的权限列表。Docker 容器限制执行范围。DM 配对码、执行审批 workflow、TLS 证书绑定——多层纵深防御。



行业的未解之题

Prompt Injection 仍是行业级未解问题。当 Agent 处理不可信输入（用户消息、网页内容、第三方数据），如何防止被“劫持”？这需要模型层 + 架构层的共同进化。

OpenClaw 给我们的启示

三个关于未来的深层判断

1

个人 AI 是基础设施，不是奢侈品

就像互联网接入、智能手机一样，个人 AI Agent 将成为现代生活的基本配置。问题不是“要不要”，而是“谁控制”。开源、本地优先的方案是确保每个人都能平等获得这一能力的关键。

2

Agent OS 将重新定义“操作系统”

未来的“操作系统”不是管理硬件的内核，而是管理“智能体”的控制平面——会话、记忆、工具、权限、调度。谁掌握了这个控制平面，谁就掌握了下一代计算的入口。

3

开源是这个时代的最大公共品

深圳政府已起草 OpenClaw 支持政策。这不是偶然，而是开源 Agent 基座设施对国家竞争力的战略意义已被认知。当 AI 能力成为基础设施，开源 = 数字主权 = 国家安全。



问题从来不是
“AI 能做什么”
而是
“我们想让 AI 为谁做事”

OpenClaw 的答案是：为每一个人。

感谢思考 · Q & A